BOOMI REFERENCE DESIGN · RESILIENT SYNCHRONOUS API

Resilient Orders API

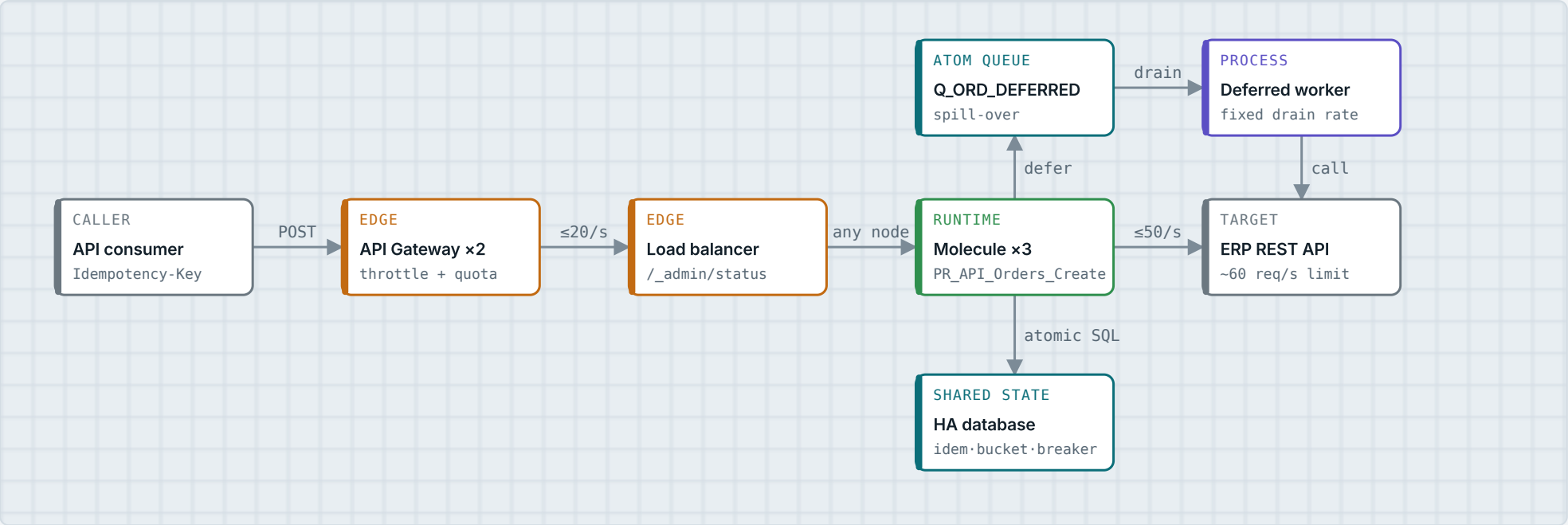
A POST /orders API built for multi-node runtimes. The gateway enforces per-consumer rate limits. An idempotency key guards against duplicate orders. A shared token bucket throttles calls to the ERP, and a circuit breaker stops calling it while it is down. Load that the ERP can't take right now spills onto an Atom Queue and drains at a fixed rate.

Start · Logic / control · Data shaping · Connector · Process call · Error handling · Stop / return

---

00 · ARCHITECTURE

Where each guarantee lives



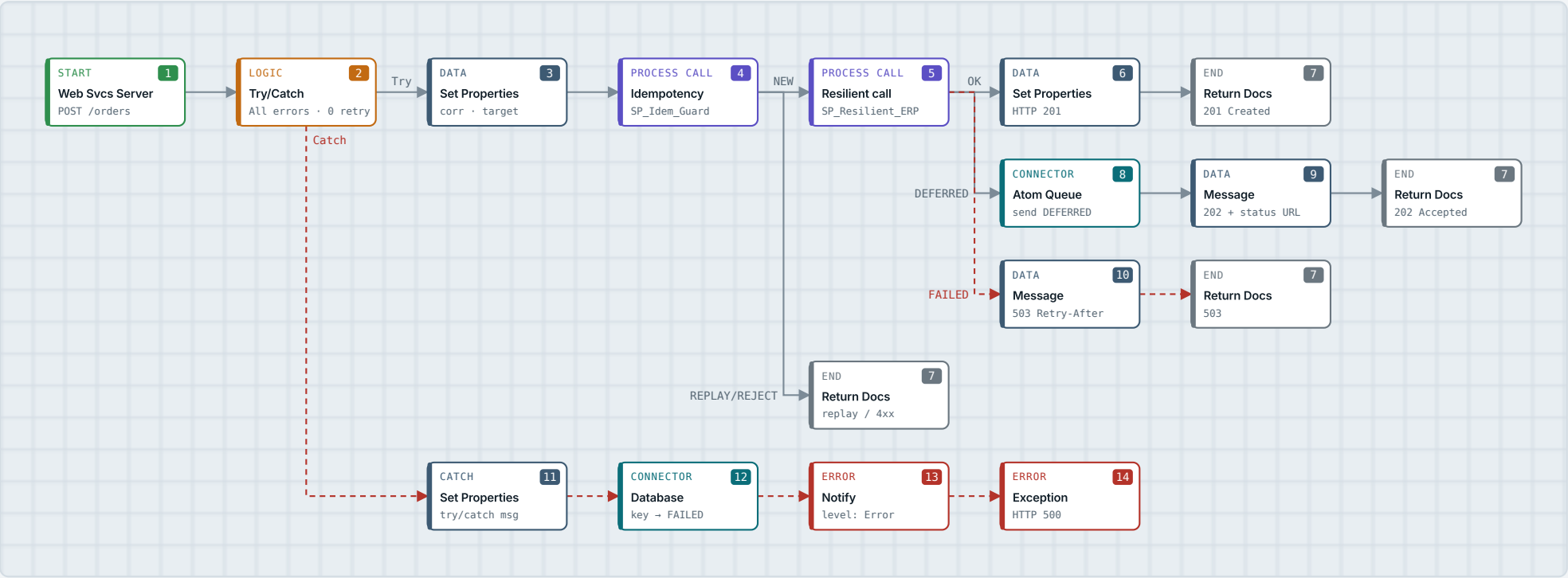
REQUIREMENT	MECHANISM	WHY IT WORKS ACROSS NODES
High availability	2+ gateway nodes, 3+ Molecule nodes behind a load balancer, HA database	Any node can serve any request; health check on `/_admin/status` removes a dead node
Idempotency	Idempotency-Key header + `claim_idem()` in the shared DB	Primary-key insert is atomic, so only one node wins a given key
Distributed state	Idempotency, token bucket and breaker all in one DB, never in node memory	Boomi caches and process properties are per node or per execution, so they can't coordinate a cluster
Rate limiting	Gateway plan: throttle (calls/sec) + quota (calls/day) per consumer key	Rejected at the edge with 429 before any runtime thread is used
Throttling	DB token bucket for the ERP + Atom Queue spill-over	One bucket shared by every node caps total ERP traffic

Circuit breaker	`cb_allow()` / `cb_record()` on a shared row	One node's failures open the circuit for all nodes; only one node wins the half-open trial

01 · PROCESS

PR_API_Orders_Create

`Process Mode: Low Latency` `Molecule ×3 or Runtime Cloud` `Allow Simultaneous Executions: On` `Purge Data Immediately: On`



1 Start — Web Services Server

START

PROPERTY	VALUE	WHY
Action / Operation	`Listen · Execute · Object: orders`	Linked from an API Service component as POST /orders
Input / Output Type	`Single Data / Single Data`	
Profiles	`JSON: ORD_Create_Request / ORD_Create_Response`	
Required header	`Idempotency-Key (UUID)`	Arrives as DDP `inheader_Idempotency-Key`

2 Try/Catch

CONTROL

PROPERTY	VALUE	WHY
Failure Trigger	`All Errors`	
Retry Count	`0`	The circuit breaker owns retries. Hidden retries here would hammer a failing ERP.

3 Set Properties — context

DATA

PROPERTY	VALUE	WHY
DDP_CORRELATION_ID	`Execution Property › Execution Id`	Returned in X-Correlation-Id
DDP_TARGET	`Static: "ERP"`	Key into bucket and breaker rows

4 Process Call — idempotency guard

EXEC

PROPERTY	VALUE	WHY
Process	`SP_Idempotency_Guard`	Section 02
Wait / Abort if fails	`On / Off`	
Return paths	`NEW · REPLAY · REJECT`	REPLAY and REJECT already carry body + status

5 Process Call — resilient call

EXEC

PROPERTY	VALUE	WHY
Process	`SP_Resilient_ERP_Call`	Section 03; the async worker reuses it
Wait / Abort if fails	`On / Off`	
Return paths	`OK · DEFERRED · FAILED`	

6 Set Properties — 201 response

DATA

PROPERTY	VALUE	WHY
HTTP status	`WSS response-code document property = 201`	Label varies by release
outheader_X-Correlation-Id	`DDP_CORRELATION_ID`	Response header via `outheader_` DDP

7 Return Documents (×5)

END

PROPERTY	VALUE	WHY
Label	(blank)	Every branch ends in a response to the caller

8 Atom Queue — spill-over

CONNECTOR

PROPERTY	VALUE	WHY
Action / Queue	Send / Q_ORD_DEFERRED (Point-to-Point)	Accept now, process when the ERP has capacity
Carried DDPs	DDP_IDEM_KEY, DDP_CORRELATION_ID, DDP_DEFER_REASON	DDPs travel with queue messages

9 Message — 202 Accepted

DATA

PROPERTY	VALUE	WHY
Body	{"status":"ACCEPTED","statusUrl":"/orders/status/{1}"}	{1} = DDP_IDEM_KEY. Quote literal braces per Message shape rules.
HTTP status	202 (Set Properties before Return)	

10 Message — 503

DATA

PROPERTY	VALUE	WHY
Body	`{"error":"UPSTREAM_UNAVAILABLE","correlationId":"{1}"}`	
outheader_Retry-After	`30`	Matches breaker open window

> The idempotency row is already marked FAILED, so the client may safely retry with the same key.

11 Set Properties — catch

CATCH

PROPERTY	VALUE	WHY
DPP_ERR_MSG	`Meta Information › Base › Try/Catch Message`	

12 Database — release key

CONNECTOR

PROPERTY	VALUE	WHY
Action / SQL	`Send · UPDATE api_idempotency SET status='FAILED' WHERE idem_key=?`	Unblocks a client retry immediately instead of after the 2-minute lock

13 Notify

ERROR

PROPERTY	VALUE	WHY
Level / Message	`Error / Corr {1}: {2}`	

14 Exception

ERROR

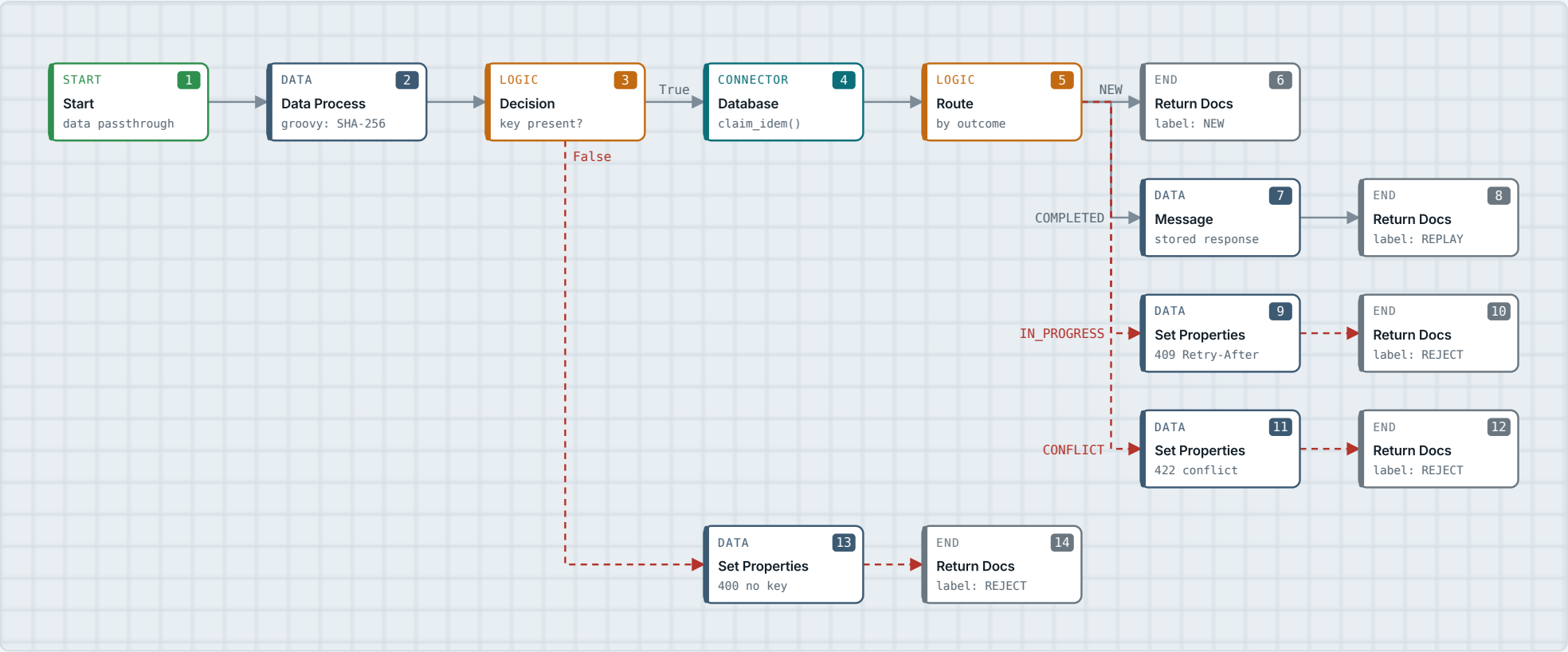
PROPERTY	VALUE	WHY
Stop a single document	`Off`	Caller receives HTTP 500

---

02 · SUB PROCESS

SP_Idempotency_Guard

`Start: Data Passthrough` `Returns: NEW · REPLAY · REJECT`



2 Data Process — key + body hash

DATA

PROPERTY	VALUE	WHY
Processing Step	Custom Scripting · Groovy 2.4	Same key with a different body must be refused

```
import java.security.MessageDigest
String P = "document.dynamic.userdefined."
for (int i = 0; i < dataContext.getDataCount(); i++) {
  InputStream is = dataContext.getStream(i)
  Properties props = dataContext.getProperties(i)
  byte[] body = is.bytes
  String hash = MessageDigest.getInstance("SHA-256")
                 .digest(body).encodeHex().toString()
  props.setProperty(P + "DDP_REQ_HASH", hash)
  props.setProperty(P + "DDP_IDEM_KEY",
      props.getProperty(P + "inheader_Idempotency-Key") ?: "")
  dataContext.storeStream(new ByteArrayInputStream(body), props)
}
```

3 Decision — key present?

CONTROL

PROPERTY	VALUE	WHY
First value	Document Property › DDP_IDEM_KEY	
Comparison / Second	Not Equal To / Static: (empty)	False → 400

4 Database — claim key

CONNECTOR

PROPERTY	VALUE	WHY
Action	`Get`	Returns one row
Profile SQL	`SELECT outcome, http_code, response_body FROM claim_idem(?, ?)`	Atomic; see section 05
Parameters	`1 = DDP_IDEM_KEY, 2 = DDP_REQ_HASH`	
Connection	`Pooled · max 20 per node · timeout 5 s`	Fail fast if the DB is unhealthy

> This replaces the request body with the DB row. Keep the original body by placing a Branch before shape 4 and passing it on Branch 2, or by caching it in a Document Cache and retrieving it on the NEW path.

5 Route — outcome

CONTROL

PROPERTY	VALUE	WHY
Route by	`Profile Element › outcome`	
Keys	`NEW · COMPLETED · IN_PROGRESS · CONFLICT`	Default route → Exception

7 Message — stored response

DATA

PROPERTY	VALUE	WHY
Body	`{1} = Profile Element › response_body`	Byte-identical replay of the first answer
HTTP status / header	`http_code · outheader_Idempotent-Replayed = true`	

9–13 Set Properties — reject codes

DATA

BRANCH	STATUS	MEANING
IN_PROGRESS	`409 + Retry-After: 2`	Same key is running on another node
CONFLICT	`422`	Key reused with a different body
No key	`400`	Idempotency-Key header is mandatory

6·8·10·12·14 Return Documents

END

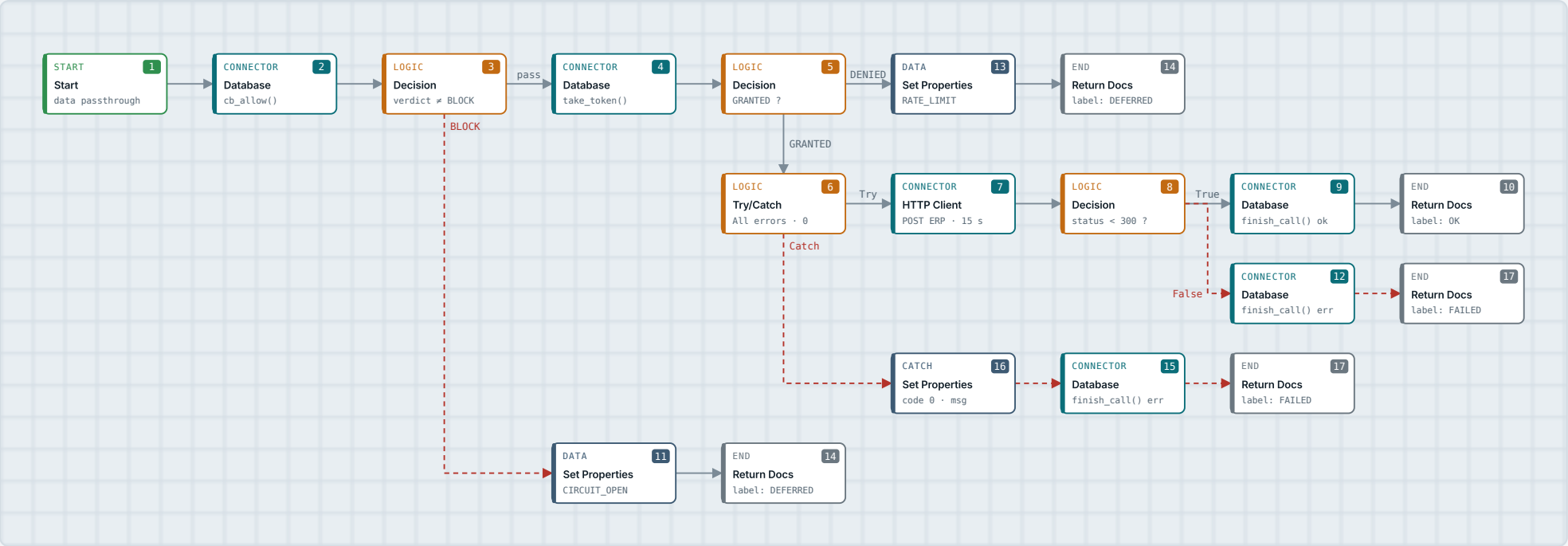
PROPERTY	VALUE	WHY
Labels	`NEW · REPLAY · REJECT`	The three REJECT shapes share one label, so they merge into one return path

---

03 · SUB PROCESS

SP_Resilient_ERP_Call

`Start: Data Passthrough` `Breaker → token → call → record` `Returns: OK · DEFERRED · FAILED`



2 Database — breaker check

CONNECTOR

PROPERTY	VALUE	WHY
Action / SQL	`Get · SELECT cb_allow(?) AS verdict`	ALLOW · TRIAL · BLOCK
Parameter	`DDP_TARGET`	

> As with the guard, use a Branch or Document Cache so the order body survives the DB lookups. Shapes 2 and 4 are only checks.

3 Decision — circuit open?

CONTROL

PROPERTY	VALUE	WHY
First / Comparison / Second	`verdict · Not Equal To · "BLOCK"`	False → defer with reason CIRCUIT_OPEN

4 Database — take token

CONNECTOR

PROPERTY	VALUE	WHY
Action / SQL	`Get · SELECT take_token(?) AS grant`	Cluster-wide token bucket
Bucket config	`capacity 50, refill 25/s`	Set to ~80% of the ERP's published limit

5 Decision — token granted?

CONTROL

PROPERTY	VALUE	WHY
First / Comparison / Second	`grant · Equal To · "GRANTED"`	False → defer with reason RATE_LIMIT

6 Try/Catch

CONTROL

PROPERTY	VALUE	WHY
Failure Trigger / Retry	`All Errors / 0`	Timeouts and connection errors count as breaker failures

7 HTTP Client — ERP

CONNECTOR

PROPERTY	VALUE	WHY
Action	`Send · POST /api/salesorders`	
Request header	`Idempotency-Key = DDP_IDEM_KEY`	Passes the guarantee downstream if the ERP supports it
Return Application Error Responses	`On`	4xx/5xx come back for the Decision
Connect / Read timeout	`3 000 / 15 000 ms`	Shorter than the gateway timeout so the caller gets a real answer

8 Decision — 2xx?

CONTROL

PROPERTY	VALUE	WHY
First value	`Meta Information › Base › Application Status Code`	
Comparison / Second	`Less Than / 300`	

9·12·15 Database — finish call

CONNECTOR

PROPERTY	VALUE	WHY
Action / SQL	`Get · SELECT finish_call(?, ?, ?, ?)`	Updates breaker and idempotency row in one transaction
Parameters	`DDP_TARGET, DDP_IDEM_KEY, status code (0 in catch), response body`	

> Only 5xx, 429 and timeouts (code 0) count as breaker failures. A 4xx is the client's fault: it's stored as COMPLETED and replayed on retry.

11·13·16 **Set Properties — reasons** DATA

PROPERTY	VALUE	WHY
DDP_DEFER_REASON	`CIRCUIT_OPEN or RATE_LIMIT`	Visible in queue and metrics
DDP_ERR_MSG (catch)	`Meta Information › Base › Try/Catch Message`	

10·14·17 **Return Documents** END

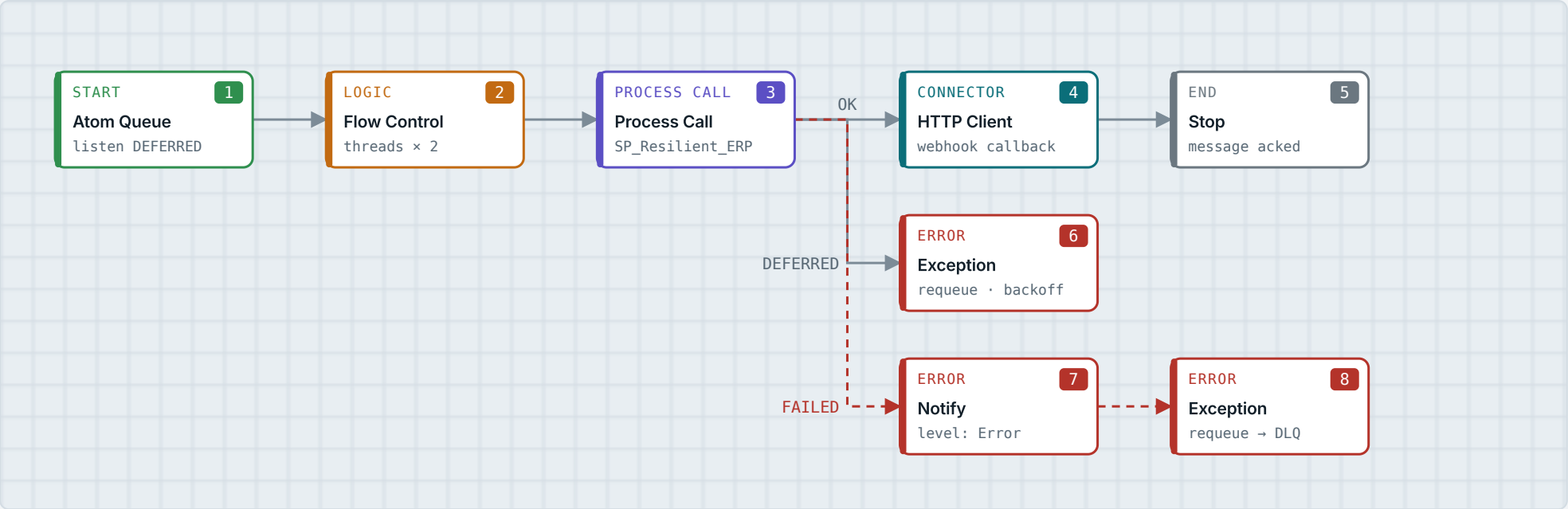
PROPERTY	VALUE	WHY
Labels	`OK · DEFERRED · FAILED`	Same-label returns merge

---

04 · PROCESS

PR_API_Orders_Deferred_Worker

Process Mode: General | Listener concurrency: 1 per node | Flow Control: 2 threads



1 Start — Atom Queue listen

START

PROPERTY	VALUE	WHY
Queue	Q_ORD_DEFERRED · Point-to-Point	
Batch size / max concurrent	10 / 1 per node	Fixed, predictable drain rate

2 Flow Control

CONTROL

PROPERTY	VALUE	WHY
Parallel Type / Units	`Threads / 2`	The token bucket still caps the total, so threads only need to keep it busy
Run Each Document Individually	`On`	One order per thread pass

3 Process Call — SP_Resilient_ERP_Call

EXEC

PROPERTY	VALUE	WHY
Wait / Abort if fails	`On / Off`	Same sub process as the sync path

4 HTTP Client — callback

CONNECTOR

PROPERTY	VALUE	WHY
Target	`Consumer webhook URL (optional)`	Otherwise the client polls the status URL, which reads api_idempotency

6·8 Exception — requeue

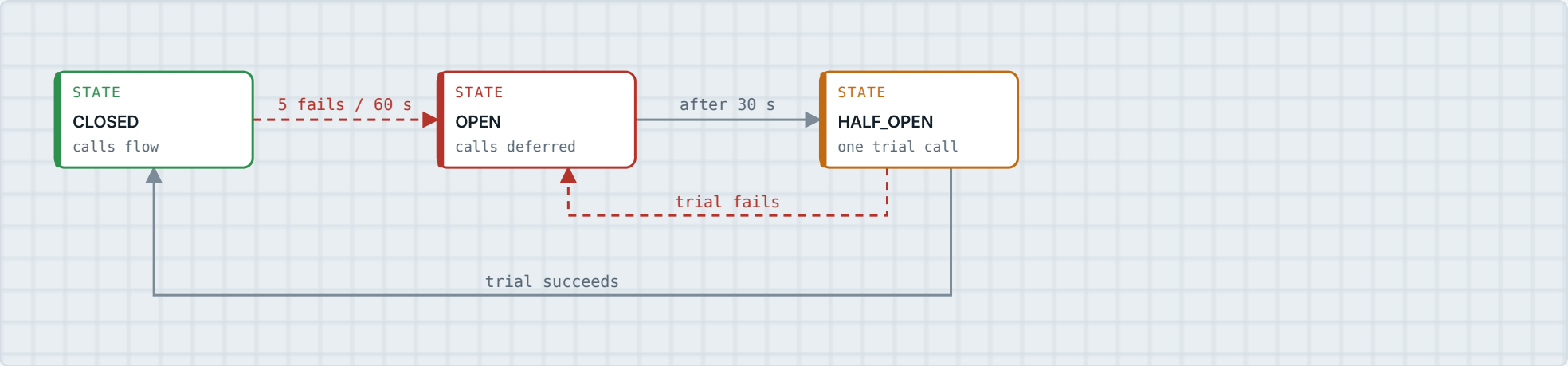
ERROR

PROPERTY	VALUE	WHY
Stop a single document	`Off`	Failing the execution makes Atom Queue redeliver the message with back-off
Message	`Deferred: {1}`	{1} = DDP_DEFER_REASON

> After the runtime's redelivery limit, Atom Queue moves the message to the queue's dead-letter queue, which you can view and resend from Queue Management. Redelivery count and back-off are runtime queue properties; confirm the defaults on your release.

05 · CIRCUIT BREAKER

Three states, one shared row



STATE	REQUEST BEHAVIOUR	LEAVES WHEN
CLOSED	Calls go through; failures are counted in a 60-second window	5 failures inside the window → OPEN
OPEN	No ERP calls. Sync requests get 202 + queue; the worker requeues.	30 s after opening, the first caller atomically flips it to HALF_OPEN
HALF_OPEN	Exactly one trial call (the node that won the flip); every other node stays blocked	Trial succeeds → CLOSED. Trial fails → OPEN for another 30 s. Trial node dies → re-trial after 30 s.

06 · SHARED STATE

Database objects (PostgreSQL)

Every cross-node decision is a single SQL statement, so row locks make it safe without any locking inside Boomi. Port to SQL Server with `MERGE` and `OUTPUT` if that's your platform.

```
-- 1. Idempotency -----------------------------------------------------
create table api_idempotency (
  idem_key      varchar(80) primary key,
  req_hash      char(64)    not null,
  status        varchar(12) not null,          -- IN_PROGRESS | COMPLETED | FAILED
  http_code     int,
  response_body text,
  locked_until  timestamptz,
  created_at    timestamptz not null default now(),
  expires_at    timestamptz not null
);

create or replace function claim_idem(p_key text, p_hash text)
returns table(outcome text, http_code int, response_body text) as $$
begin
  insert into api_idempotency(idem_key, req_hash, status, locked_until, expires_at)
  values (p_key, p_hash, 'IN_PROGRESS', now() + interval '2 minutes', now() + interval '24 hours')
  on conflict (idem_key) do nothing;
  if found then return query select 'NEW'::text, null::int, null::text; return; end if;

  -- same body, previous attempt failed or its node died: take it over
  return query
  update api_idempotency i
     set status = 'IN_PROGRESS', locked_until = now() + interval '2 minutes'
   where i.idem_key = p_key and i.req_hash = p_hash
     and (i.status = 'FAILED' or (i.status = 'IN_PROGRESS' and i.locked_until < now()))
  returning 'NEW'::text, null::int, null::text;
  if found then return; end if;

  return query
  select case when i.req_hash <> p_hash   then 'CONFLICT'
              when i.status = 'COMPLETED' then 'COMPLETED'
              else 'IN_PROGRESS' end,
         i.http_code, i.response_body
    from api_idempotency i where i.idem_key = p_key;
```

```
end $$ language plpgsql;

-- 2. Token bucket ------------------------------------------------------
create table api_rate_bucket (
  target varchar(40) primary key, capacity int not null,
  refill_per_sec numeric not null, tokens numeric not null,
  updated_at timestamptz not null default clock_timestamp()
);
insert into api_rate_bucket values ('ERP', 50, 25, 50, clock_timestamp());

create or replace function take_token(p_target text) returns text as $$
begin
  update api_rate_bucket b
     set tokens = least(b.capacity, b.tokens + extract(epoch from clock_timestamp() - b.updated_at) * b.refill_per_sec) - 1,
         updated_at = clock_timestamp()
   where b.target = p_target
     and least(b.capacity, b.tokens + extract(epoch from clock_timestamp() - b.updated_at) * b.refill_per_sec) >= 1;
  return case when found then 'GRANTED' else 'DENIED' end;
end $$ language plpgsql;

-- 3. Circuit breaker ---------------------------------------------------
create table api_circuit (
  target varchar(40) primary key,
  state varchar(10) not null default 'CLOSED',    -- CLOSED | OPEN | HALF_OPEN
  failures int not null default 0, window_start timestamptz, opened_at timestamptz,
  threshold int not null default 5, open_seconds int not null default 30
);
insert into api_circuit(target) values ('ERP');

create or replace function cb_allow(p_target text) returns text as $$
declare s text;
begin
  -- one caller wins the flip to HALF_OPEN; a stuck trial is re-armed after open_seconds
  update api_circuit set state = 'HALF_OPEN', opened_at = clock_timestamp()
   where target = p_target and state in ('OPEN', 'HALF_OPEN')
     and opened_at + make_interval(secs => open_seconds) <= clock_timestamp();
  if found then return 'TRIAL'; end if;
  select state into s from api_circuit where target = p_target;
  return case s when 'CLOSED' then 'ALLOW' else 'BLOCK' end;
end $$ language plpgsql;

create or replace function cb_record(p_target text, p_ok boolean) returns void as $$
begin
  if p_ok then
```

```
    update api_circuit set state = 'CLOSED', failures = 0, window_start = null where target = p_target;
  else
    update api_circuit
       set failures     = case when window_start is null or window_start < clock_timestamp() - interval '60 seconds' then 1 else failures + 1 end,
           window_start = case when window_start is null or window_start < clock_timestamp() - interval '60 seconds' then clock_timestamp() else
window_start end
     where target = p_target;
    update api_circuit set state = 'OPEN', opened_at = clock_timestamp()
     where target = p_target and (state = 'HALF_OPEN' or failures >= threshold);
  end if;
end $$ language plpgsql;

-- 4. Finish: breaker + idempotency in one transaction -----------------
create or replace function finish_call(p_target text, p_key text, p_code int, p_body text)
returns text as $$
declare upstream_fault boolean := (p_code = 0 or p_code = 429 or p_code >= 500);
begin
  perform cb_record(p_target, not upstream_fault);
  update api_idempotency
     set status = case when upstream_fault then 'FAILED' else 'COMPLETED' end,
         http_code = p_code, response_body = p_body, locked_until = null
   where idem_key = p_key;
  return case when upstream_fault then 'FAILED' else 'OK' end;
end $$ language plpgsql;

-- 5. Housekeeping (scheduled Boomi process, every 15 min) -------------
delete from api_idempotency where expires_at < now();
```

> The token bucket serialises on one row. That is fine up to a few hundred grants per second. Beyond that, move the bucket and breaker to Redis (called through the HTTP Client or a Redis connector) and keep idempotency in the DB.

---

07 · RATE LIMITING VS THROTTLING

Settings at each layer

LAYER	SETTING	VALUE / EFFECT

Gateway plan (per consumer key)	`Throttle (calls/sec) + Quota (calls/day)`	e.g. 20/s and 100 000/day → 429 with Retry-After at the edge
Gateway	`Max payload size, backend timeout`	1 MB; 20 s (longer than the ERP read timeout + DB time)
Load balancer	`Health check` `GET /_admin/status` `per node`	Unhealthy node is drained within seconds
Runtime Shared Web Server	`Max listener threads per node`	Caps in-flight requests; excess waits at the LB instead of exhausting heap
Process	`Low Latency mode`	No per-call JVM fork; no document tracking
ERP token bucket	`capacity / refill`	Hard cluster-wide ceiling on ERP calls
Deferred queue	`listener concurrency × Flow Control threads`	Drains backlog without starving live traffic of tokens

- Node failover mid-request: the client times out and retries with the same key. `claim_idem` returns IN_PROGRESS until the 2-minute lock lapses, then hands the key to the retry, so no duplicate order is created.
- If the ERP accepts an Idempotency-Key header, forward it. That closes the last gap: a node that dies after the ERP commits but before `finish_call`.
- Alert on `api_circuit.state <> 'CLOSED'`, Q_ORD_DEFERRED depth, and gateway 429 rate per consumer.